

Project 1 Complete Report

1. Introduction

The domain selected for this project is the $2 \times 2 \times 2$ Rubik's Cube, also called a Pocket Cube, a 3-dimensional classic combinatorial puzzle consisting of 8 corner pieces and 24 visible sticker faces. The goal of the puzzle is to restore a scrambled cube configuration to a uniform color distribution on all six faces through a sequence of face rotations. The goal of this project is to use classical state-space search algorithms to find those sequences.

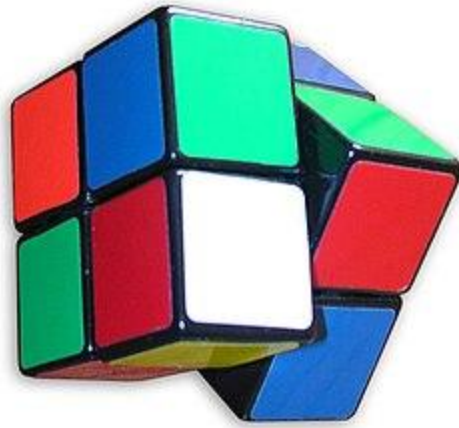


Fig. 1. An example of a scrambled Pocket Cube. [1]

2. Problem Definition

To model the Pocket Cube as a formal state-space search problem, the system is defined as a deterministic, discrete, and fully observable environment represented by the formal 6-tuple (S, s_0, A, T, G, c) :

State Representation (S)

A state $s \in S$ is represented as a static 24-element tuple of integers:

$$s = (c_0, c_1, c_2, \dots, c_{23})$$

Each index corresponds to a specific sticker position on the standard 2x2 net layout. The integer values $c_i \in \{0,1,2,3,4,5\}$ denote the face colors mapped as follows:

- 0: White (Up face)
- 1: Green (Left face)
- 2: Red (Front face)
- 3: Blue (Right face)
- 4: Orange (Back face)
- 5: Yellow (Down face)

The face colors are stored with the top face first, then the upper ring around the sides, then the lower ring, and finally the bottom face. For example, the solved configuration of the cube is:

$$s_{\text{solved}} = (0,0,0,0, 1,1,2,2,3,3,4,4, 1,1,2,2,3,3,4,4, 5,5,5,5)$$

Cube stickers								Array subscripts							
W W Up W W								00 01 Up 02 03							
G G	R R	B B	O O					04 05	06 07	08 09	10 11				
Left	Front	Right	Back					Left	Front	Right	Back				
G G	R R	B B	O O					12 13	14 15	16 17	18 19				
Y Y Down Y Y								20 21 Down 22 23							

Fig. 2. Visual representation of the state tuple. [2]

Initial State (s_0)

The initial scrambled state $s_0 \in S$ specifies the starting configuration of the cube prior to running the search algorithm. It is generated by executing a sequence of k randomized valid moves starting from s_{solved} . To achieve effective randomization, at each step the inverse of the move just executed is temporarily disabled to prevent immediate cancellation (e.g., executing 'U' bans 'u' on the next turn; see Actions explanation below).

Actions (A)

The set of valid actions $A(s)$ available from any state s consists of quarter-turn rotations restricted to three primary faces (Up, Right, Front) in both clockwise and counterclockwise directions, denoted using a modified version of Singmaster notation with lowercase letters

for the “prime” turns as described in [2]. Although there are many other possible rotations that can be performed on the Pocket Cube, the geometry of the puzzle makes it possible to solve it using only these six basic actions [2]:

$$A = \{'U', 'u', 'R', 'r', 'F', 'f'\}$$

- 'U' / 'u': Clockwise / Counter-clockwise (90°) turn of the Up face.
- 'R' / 'r': Clockwise / Counter-clockwise (90°) turn of the Right face.
- 'F' / 'f': Clockwise / Counter-clockwise (90°) turn of the Front face.

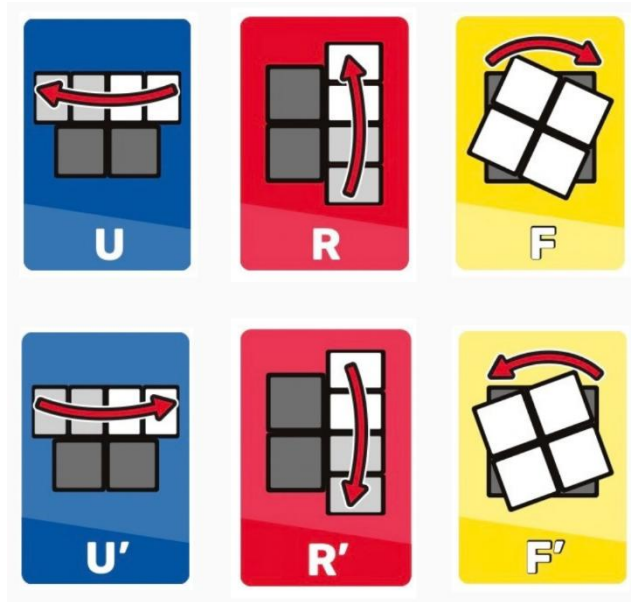


Fig. 3. Visual representations of the 6 basic turns. Adapted from [3].

Transition Model (T)

The transition function $T: S \times A \rightarrow S$ deterministically computes the successor state $s' = \text{RESULT}(s, a)$ when applying action $a \in A$ to state s . Each action permutes 12 specific indices within the 24-element tuple according to the 3D mechanics of a face turn while leaving the remaining 12 indices unchanged. The implementation of the logic for this was directly adapted from the source code provided in [2].

Goal Test (G)

The goal test is an explicit Boolean function evaluating whether a given state s matches the solved target state:

$$\text{GOAL}(s) = \begin{cases} \text{True} & \text{if } s = s_{\text{solved}} \\ \text{False} & \text{otherwise} \end{cases}$$

Equivalently, it checks that all four stickers on each of the cube's six faces share identical color integer values.

Path-Cost Function (c)

Every single action (quarter turn) incurs a uniform step cost:

$$c(s, a, s') = 1$$

Thus, the path cost $g(n)$ for reaching node n via some sequence of moves is simply equal to the total length of the sequence. An optimal solver seeks a path that minimizes $g(n)$, finding the shortest move sequence required to return the cube to s_{solved} .

3. Search Algorithms

Breadth-First Search (BFS)

BFS is an uninformed search strategy that expands nodes systematically level-by-level using a FIFO queue. It maintains a visited set to store explored states and prevent redundant path evaluation. Because each quarter-turn move carries an identical cost of 1, BFS guarantees finding the optimal (shortest move-count) solution.

Depth-First Search (DFS)

DFS is an uninformed search strategy implemented recursively that explores as deep as possible along each branch before backtracking. The implementation in this project was modified to use a `path_visited` set tracking current call-stack states to prevent cycle loops without blocking re-entry from alternative paths. To avoid exploring infinitely deep trees, it also includes a `max_depth` threshold (converting it to Depth-Limited Search), which was set at 15 throughout the experiment, since “God’s Number” (the length of the best possible solution from any state) for the Pocket Cube is 14 [4] (1 was added for redundancy). DFS does not guarantee optimal solution lengths and is highly sensitive to depth limits.

A* Search

A* is an informed search strategy that prioritizes node expansion using the evaluation function $f(n) = g(n) + h(n)$, managed via a min-heap priority queue.

- $g(n)$: Exact path cost from the start state (number of moves taken).
- $h(n)$: Heuristic estimate calculated as the number of mismatched sticker color positions relative to the goal state (i.e. Hamming distance) divided by 8 (the maximum number of color placements affected per move).

Because $h(n)$ is admissible (never overestimates actual remaining moves) and consistent, A* guarantees finding an optimal solution while exploring significantly fewer nodes than uninformed approaches.

4. Planning Model

Action Preconditions

Every move in $A = \{'U', 'u', 'R', 'r', 'F', 'f'\}$ can be applied to any valid state $s \in S$. There are no physical constraints preventing a turn. The only restriction is during scrambling (see the Initial State section above).

Action Effects

Executing a move permutes exactly 12 of the 24 array indices in state vector s :

- The 4 stickers on the turned face rotate among themselves.
- 8 stickers from neighboring faces shift along the outer ring.
- The remaining 12 stickers stay in their original positions.

Why Action Order Matters

Order matters because adjacent faces share physical pieces. For example, the U (Up) and R (Right) faces share two corner pieces.

Executing 'U' first physically moves new stickers into the R face's layer. When you follow up with an 'R' turn, it grabs those newly arrived stickers and shifts them. If you reverse the sequence and execute 'R' first, 'R' operates on the original stickers in its layer instead, since 'U' has not moved the new ones into position yet.

Because each turn changes which stickers sit in the path of intersecting faces, swapping the sequence alters which stickers get moved downstream, resulting in completely different end configurations.

5. Testing and Results

The algorithms were evaluated across eight different scramble depths (0, 1, 3, 5, 10, 15, 25, and 100 moves) using a fixed random seed (42). As a bonus, DFS was also run without a depth limit on a cube with scramble depth 15.

Performance Summary

Solution Discovery

Breadth-First Search (BFS), A* search, and Depth-Limited DFS (capped at depth 15) successfully found a solution in every test case. Unconstrained DFS failed to complete, triggering a RecursionError after exceeding Python's maximum call stack limit.

Lowest-Cost Solution

BFS and A* consistently tied for the lowest-cost solutions across all test cases (e.g., 1 move for depth 1, 5 moves for depth 5, and 10 moves for depth 100). Depth-Limited DFS returned highly sub-optimal paths, costing 14 or 15 moves even on simple 1-turn scrambles.

Shortest vs. Least Expensive

Because every turn has a uniform step cost of 1, the shortest move sequence is identical to the least expensive solution. BFS and A* both consistently found this optimal path.

Node Expansions

A* expanded the fewest nodes in every single run. At scramble depth 5, A* expanded 73 nodes compared to BFS's 551. At depth 100, A* expanded 77,636 nodes compared to BFS's 303,549. Depth-Limited DFS expanded by far the most nodes, expanding over 70 million nodes during the depth-15 test alone.

Memory Usage

Unconstrained DFS consumed the most call stack memory, leading to a stack overflow crash. Among the successful runs, BFS required the most memory because its queue holds every unexplored state across an entire depth level. A* required less memory because its guided frontier stays significantly smaller.

Overall Efficiency

A* was the most efficient algorithm. It guaranteed an optimal path every time while searching only a small fraction of the states that BFS had to check.

Algorithm Behavior Analysis

BFS searches blindly in expanding rings. It checks every single 1-move state, then every 2-move state, and so on. While this guarantees finding the shortest path, it wastes memory and computation expanding unhelpful branches.

A* uses the sticker mismatch heuristic to estimate how close a cube is to being solved. This acts as a guide, steering the search toward promising moves and ignoring bad ones. As a result, it finds the optimal path while exploring far fewer total states.

DFS blindly follows a single branch as deep as allowed (15 moves) before backing up. Because it lacks any sense of direction or path cost, it wanders down long, useless move chains—yielding 14-to-15-move solutions for simple scrambles and expanding millions of unnecessary nodes. Without an artificial depth cap, it gets stuck in endless loops until Python runs out of memory.

Selected Results

The following metrics were recorded for a cube with scramble depth 15:

Algorithm	Solution Found?	Solution Length/Cost	Expanded Nodes	Runtime (seconds)
BFS	Yes	11	628,695	10.81
DFS (depth limit = 15)	Yes	15	70,004,192	110.66
A*	Yes	11	260,818	11.29
DFS (no depth limit)	No	N/A	Unknown	0.02 before crashing

Raw Output

=====

Testing with scramble depth = 15

Current cube state: ROOGWGWYRWBBGBYOYGWORBYR

R O

O G

W G W Y R W B B

G B Y O Y G W O

R B

Y R

Scramble moves: ['u', 'F', 'F', 'u', 'R', 'R', 'R', 'u', 'F', 'u', 'u', 'r', 'F', 'r', 'f']

Solving with BFS...

BFS solution: ['F', 'R', 'f', 'R', 'U', 'F', 'F', 'r', 'F', 'u', 'f']

Cost: 11

Nodes expanded: 628695

Time: 10.614166498184204

Solving with DFS (depth limit 15)...

DFS solution: ['F', 'F', 'F', 'U', 'R', 'F', 'R', 'u', 'f', 'U', 'U', 'F', 'u', 'r', 'F']

Cost: 15

Nodes expanded: 70004192

Time: 111.82055854797363

Solving with A*...

A* solution: ['u', 'F', 'r', 'u', 'r', 'F', 'u', 'F', 'u', 'r', 'U']

Cost: 11

Nodes expanded: 260818

Time: 10.227304697036743

6. Discussion

Algorithm Strengths and Limitations

Breadth-First Search (BFS):

Strengths: Guarantees an optimal, minimal-move solution on unweighted graphs because it explores state space level-by-level.

Limitations: Memory usage grows exponentially $O(b^d)$ with depth. As the scramble distance increases, the FIFO queue must store every state at the current depth layer, rapidly consuming system RAM.

Depth-First Search (DFS):

Strengths: Requires minimal memory per branch, $O(b \cdot m)$, as it only tracks nodes along the active path.

Limitations: Fails to guarantee optimal path lengths. When bounded by a depth limit of 15, it returns long 14-to-15 move solutions even for 1-turn scrambles. When unconstrained, it traverses infinitely deep branches, exceeding Python's default call stack limit and triggering a RecursionError.

A Search:*

Strengths: Retains the path optimality of BFS while drastically reducing the search space. It prioritizes states closest to the goal using the evaluation function $f(n) = g(n) + h(n)$.

Limitations: Performance relies heavily on heuristic quality. Like BFS, A* must retain open nodes in a priority queue, meaning memory can still become a bottleneck on very deep or complex configurations.

Effect of the Heuristic

Dividing the number of mismatched sticker positions by 8 provides an admissible heuristic because a single quarter-turn can fix at most 8 color mismatches relative to solved face targets. This estimate directs the priority queue toward states with better face-color alignment and prunes unpromising branches early:

- At scramble depth 5, A* expanded 73 nodes compared to 551 nodes for BFS (an 86.7% reduction).
- At scramble depth 100, A* expanded 77,636 nodes compared to 303,549 nodes for BFS (a 74.4% reduction).

By guiding node expansion, the heuristic allows A* to reach the optimal solution while processing only a fraction of the search tree required by uninformed BFS.

Time and Memory Observations

Execution Time

DFS's deep searches along suboptimal paths caused it to take much longer to finish on average than either of the other two algorithms; for example, at scramble depth 5, DFS took almost 23 seconds to find a solution, whereas BFS and A* both finished in less than 0.1. A* and BFS tended to have similar execution times, with BFS often being slightly faster, although it was quite variable.

Memory

BFS exhibited the largest operational memory footprint because its queue holds every unexplored node across the current depth level before proceeding to the next. A* kept memory usage substantially lower than BFS because the heuristic restricts frontier growth along unpromising paths. Recursive DFS uses minimal heap memory for state storage, but

consumes call-stack memory proportional to tree depth. Without depth constraints, stack memory rapidly exhausted system limits.

Implementation Limitations

Object Instantiation Overhead: Successor generation relies on instantiating temporary PocketCube objects to calculate state transitions, creating allocation overhead during node expansion in Python.

Heuristic Blindness: The sticker mismatch heuristic treats sticker positions independently rather than accounting for rigid 3D corner piece geometry. Because individual stickers cannot move independently of their piece, the heuristic underestimates actual remaining moves on complex scrambles.

Single-Threaded Processing: Algorithms run synchronously on a single core, limiting performance compared to compiled multi-threaded or bitboard implementations.

7. Conclusion

This project evaluated the practical trade-offs among search strategies when applied to the 2x2 Rubik's Cube state space. Both Breadth-First Search (BFS) and A* search consistently returned optimal, minimum-cost move sequences across all test scrambles. However, A* dramatically outperformed BFS in operational efficiency: by using an admissible sticker-mismatch heuristic, A* reduced node expansions by up to 86.7% compared to BFS while guaranteeing path optimality. Conversely, Depth-First Search proved unsuited for this domain; depth-limited DFS returned sub-optimal solution paths (costing 14–15 moves even for 1-turn scrambles), and unconstrained DFS failed entirely due to stack overflow recursion errors.

Key Lessons for Search and Planning

- **Heuristics mitigate combinatorial explosion:** Domain-specific knowledge, even a basic count of mismatched stickers, prunes massive portions of the search space, turning an intractable memory hog into an efficient search.
- **Algorithmic constraints must match domain structure:** Uninformed strategies like BFS become memory-bound as depth increases. Meanwhile, depth-first strategies fail in cyclic, order-dependent state spaces unless paired with strict pruning or iterative deepening.
- **Sequence order drives state reachability:** Because Rubik's Cube moves do not commute (turning 'U' then 'R' yields a different outcome than 'R' then 'U'), search

trees cannot treat actions as unordered sets. Algorithms must evaluate exact, ordered sequences to navigate the state space effectively.

Acknowledgements

Vast portions of this project (including this analysis paper) were completed using Google Gemini 3.6 Thinking, with human editing for clarity.

All project files and code can be found on GitHub here: <https://github.com/Alec-Pike/2x2Rubik-sCubeSolver>

References

- [1] M. Gonzalez. *Pocket cube twisted*. 2005. Accessed: Oct. 5, 2026. [Photo]. Available: https://commons.wikimedia.org/wiki/File:Pocket_cube_twisted.jpg
- [2] Tim. “Generating the Full List of Valid States of a 2×2 Rubik’s Cube.” Limejuice. <http://tim.id.au/limejuice/generating-the-full-list-of-valid-states-of-a-2x2-rubiks-cube/> (accessed Oct. 5, 2026)
- [3] *Rubik's 2x2 Mini Solution Guide*, Rubik's Brand Ltd., 2020. [Online]. Available: https://web.archive.org/web/20260818222624/https://cdn.prod.website-files.com/6595ca03bcd68f311fd41872/65afebcae0a3534d92a9b921_Rubiks_Solution_Guide_2x2-Mini.pdf (accessed Oct. 5, 2026).
- [4] J. Scherphuis, “Mini Cube, the 2x2x2 Rubik's Cube,” *Jaap's Puzzle Page*. [Online]. Available: <https://www.jaapsch.net/puzzles/cube2.htm> (accessed Oct. 6, 2026).